AI 时代商业工程 · 课题研究系列
BUSINESS ENGINEERING IN THE AI ERA · RESEARCH STUDY SERIES

No. 07 / 12

课题研究报告 · RESEARCH STUDY

推理工程：模型如何成为可靠且经济的服务

Inference Engineering: How a Model Becomes a Dependable, Economical Service

关于服务目标、工作记忆与利用率的课题研究

A research study on serving targets, working memory and utilisation

研究底本 SOURCE
Cuofano, G. Inference Engineering (2026)

版本 EDITION
2026 年 10 月 · October 2026

语言 LANGUAGE
中英双语 · Bilingual Chinese–English

摘要 *Abstract*

2026 年，人工智能的大部分支出已从训练转向推理。本研究以 Cuofano《推理工程》（2026）为分析底本，追问推理服务的单位成本由什么决定、各项工程杠杆在何种条件下有效、以及应如何度量与治理推理服务。研究将底本的机制、案例与 2026 年第三季度的测量数据重组为「物理—站点—机队—经济—治理」五层框架，并以五项可证伪假设组织证据。**研究发现**有三：第一，解码受限于内存带宽而非算力，**键值缓存**而非模型权重决定批处理上限，而批处理决定成本；第二，在代理工作负载中，**缓存命中率**是首要成本杠杆，同一任务的成本可因前缀是否稳定而相差四倍以上；第三，每词元价格只是吞吐—时延曲线上的一个点，单位成本由**时延目标**、**利用率**与**服务模式**共同决定，专用算力只有在利用率超过随目标收紧而上升的交叉点时才划算。一个虚构企业案例显示，仅靠缓存设计、服务模式与经套件验证的模型替换，月度模型支出即可下降约 68%。研究还把十类失效归结为「以机器而非答案为度量」，讨论了数据时效与示意算例带来的局限，并提出未来研究议程。

By 2026 most AI spending had shifted from training to inference. Using Cuofano's Inference Engineering (2026) as its source volume, this study asks what determines the unit cost of an inference service, under which conditions engineering levers work, and how the service should be measured and governed. It reorganises the source's mechanisms, cases and Q3 2026 measurements into a five-layer framework — physics, stations, fleet, economics, governance — and organises evidence around five falsifiable hypotheses. **Three findings** emerge. First, decode is bound by memory bandwidth, not arithmetic, so the **key–value cache**, not the weights, sets the ceiling on batching, and batching sets cost. Second, for agent workloads the **cache hit rate** is the primary lever: the same task can cost over four times more when its prefix is unstable. Third, a price per token is one point on a throughput–latency curve; unit cost is set jointly by **latency target**, **utilisation** and **serving mode**, and dedicated capacity pays only above a crossover that rises as targets tighten. An invented case shows monthly model spend falling about 68% through cache design, serving modes and a suite-tested model change alone. The study traces ten failure patterns to measuring the machine instead of the answer, discusses limits from dated data and illustrative arithmetic, and proposes a research agenda.

关键词 推理工程；键值缓存；缓存命中率；有效吞吐；利用率交叉点；服务模式

KEYWORDS inference engineering; key–value cache; cache hit rate; goodput; utilisation crossover; serving modes

研究问题 *Research Questions*

RQ1 推理服务的单位成本由哪些约束与决策决定，自建与按词元付费的边界在哪里？

Which constraints and decisions set the unit cost of inference, and where does the build-versus-buy boundary lie?

RQ2 各项服务技术在何种条件下有效、何时失效？

Under which conditions do serving techniques pay off, and when do they stop?

RQ3 应以哪些指标度量推理服务，才能避免「机器很忙、答案很差」？

Which measures prevent a busy fleet from producing poor answers?

目录 *Contents*

01 研究背景与问题提出
Background and Problem Statement 4

02 核心概念与理论框架
Core Concepts and Analytical Framework 6

03 研究设计与方法
Research Design and Method 9

04 发现一：工作记忆而非权重决定批处理上限
Finding 1: Working Memory, Not Weights, Sets the Ceiling on Batching 11

05 发现二：缓存命中率是代理时代的首要成本杠杆
Finding 2: Cache Hit Rate Is the Primary Cost Lever of the Agent Era 14

06 发现三：单位成本取决于目标、利用率与服务模式
Finding 3: Unit Cost Depends on Target, Utilisation and Serving Mode 17

07 案例研究与比较分析
Case Study and Comparative Analysis 20

08 讨论：失效模式、局限与风险
Discussion: Failure Modes, Limitations and Risks 23

09 结论与未来研究方向
Conclusions and Future Research Agenda 25

— 参考文献与研究说明
References & Research Notes 27

第 1 章 · CHAPTER 1

01 研究背景与问题提出

Background and Problem Statement

推理已从运维细节变成应用人工智能的主要工程问题；成本与速度如今在服务层被决定。

Inference has moved from an operational detail to the main engineering problem of applied AI; cost and speed are now decided in the serving layer.

中文

ENGLISH

1.1 词元是被「制造」出来的

Tokens Are Manufactured

底本的出发点是一个朴素的类比：模型产出的每一个词元都是被制造出来的。它在按小时计费的机器上生产，原料是驻留于内存中的模型——而内存往往比机器的算力更昂贵——交付对象则是另一端设定了时限的人或代理。据此，底本将**推理工程**界定为：以最低的「单位有效结果成本」按时制造词元的学科。在大模型短暂的历史中，这项工作长期被视为模型厂商或云厂商的「管道活」；而到 2026 年，人工智能的大部分支出发生于此，决定账单的大多数决策也由在模型之上构建产品的人而非造模型的人做出。

The source starts from a plain analogy: every token a model emits is manufactured. It is produced on a machine billed by the hour, from a model held in memory that costs more than the machine's arithmetic, for a person or agent waiting against a deadline. The source therefore defines **inference engineering** as the discipline of manufacturing tokens to their deadlines at the lowest cost per useful result. For most of the short history of large models this was treated as plumbing, the model makers' or the cloud's problem. By 2026 it is where most AI money is spent, and most decisions that set the bill are made by those who build on models, not those who make them.

1.2 一年之内的六个事实

Six Facts Within a Year

底本列举了在约一年内相继出现的六个事实，它们共同把推理推到了舞台中央：其一，调用量爆发；其二，单任务价格下降而单任务词元数上升；其三，代理把推理变成了「重复阅读」问题；其四，各大 API 的价格收敛为一张网格；其五，硬件按训练与推理一分为二；其六，服务软件栈走向成熟。本研究认为，这六点可归并为两股力量——**需求侧的词元膨胀**与**供给侧的工程成熟**——二者叠加，使服务层成为成本与速度的真正决定点。

The source lists six facts that landed within roughly a year and together moved inference to centre stage: volume exploded; the price of a task fell while tokens per task rose; agents turned inference into a re-reading problem; API prices converged on a grid; hardware split into training and inference lines; and the serving stack matured. This study groups the six into two forces — **token inflation on the demand side** and **engineering maturity on the supply side** — whose combination makes the serving layer the real point at which cost and speed are decided.

3.2P+
Google 月处理词元（千万亿）
Google tokens per month, quadrillions

7×
同比增长倍数
year-on-year growth

375+
年用量超万亿词元的云客户
cloud customers over 1T tokens/yr

~100:1
生产级代理输入输出比
agent input-to-output ratio

来源 Source: Cuofano (2026), citing Google I/O (May 2026) and builders of production agents; Q3 2026 reading.

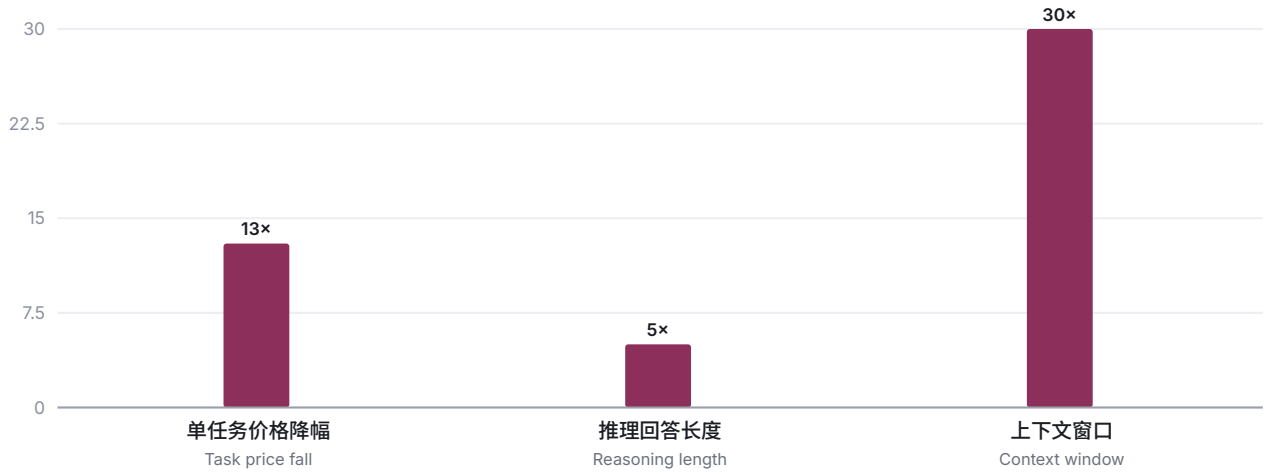
最具反直觉意味的是价格与账单的背离。按完成固定任务的成本计算，2023—2026 年「智能」的价格每年下降约

The most counter-intuitive pattern is the divergence of price and bill. Measured by the cost of completing a

13 倍；但同期推理模型的回答长度每年增长约 5 倍，上下文窗口每年扩大约 30 倍。在许多工作负载中，每项任务所消耗的词元增长得比单价下降得更快，于是账单不降反升。底本开篇提到的那位财务伙伴之问——「为什么每词元价格减半，账单却翻了一倍」——正是本研究要回答的核心困惑。

fixed task, the price of intelligence fell about thirteenfold a year from 2023 to 2026; yet reasoning answers grew about fivefold a year in length and context windows about thirtyfold. In many workloads tokens per job rose faster than price per token fell, and bills went up. The question the source attributes to a finance partner — why the bill doubled when the price per token halved — is the central puzzle this study sets out to answer.

图 1-1 需求侧的三个年度倍数（2023—2026）
Figure 1-1 *Three annual multipliers on the demand side (2023–2026)*



来源 Source: Cuofano (2026), citing Epoch AI; approximate annual rates as read in Q3 2026.

1.3 研究问题与本文结构
Research Questions and Structure

据此，本研究提出三个研究问题（见摘要页）：单位成本由什么决定、自建与采购的边界在哪里；各项技术的适用条件；以及应以何种指标度量服务。第二章建立概念与框架，第三章说明研究设计与假设；第四至六章报告三项核心发现；第七章以虚构企业案例和服务商版图做比较分析；第八章讨论失效模式、局限与风险；第九章总结并提出研究议程。

The study therefore poses three research questions (see the abstract page): what determines unit cost and where the build-versus-buy boundary lies; under which conditions each technique works; and which measures should govern the service. Chapter 2 sets out concepts and the framework; Chapter 3 the design and hypotheses; Chapters 4 to 6 report three core findings; Chapter 7 offers a case study and a comparison of the provider landscape; Chapter 8 discusses failures, limits and risks; Chapter 9 concludes with a research agenda.

核心洞见 KEY INSIGHT

问题的本质 *The heart of the problem*

推理成本问题并非「价格问题」，而是「架构问题」：同一模型、同一答案，其成本可以因请求的构造方式、所选服务模式与时延目标而相差数倍。

The cost of inference is not a pricing problem but an architectural one: the same model and the same answer can cost several times more or less depending on how requests are built, which serving mode is chosen and what latency target is set.

第 2 章 · CHAPTER 2

02 核心概念与理论框架

Core Concepts and Analytical Framework

两个阶段、一块工作记忆、四个数字——推理工程的全部技术都是围绕这三组概念展开的。

Two phases, one working memory, four numbers: every technique in inference engineering is organised around these three sets of ideas.

中文　　ENGLISH

2.1 两个阶段，两种瓶颈

Two Phases, Two Bottlenecks

每一次请求都包含两个阶段。**预填充**（prefill）并行读取全部输入词元，生成对话的工作记忆和第一个输出词元；它对每个搬运的字节做大量运算，是**算力受限**的，其时长就是用户看到第一个字之前的等待。**解码**（decode）则逐个写出答案：每生成一个词元，机器都必须从高带宽内存中读取全部权重和该对话的工作记忆，只做少量运算——它是**内存带宽受限**的，运算单元大部分时间在等数据。底本认为，几乎所有后续技术都是对这一差异的回应。

Every request has two phases. **Prefill** reads all input tokens in parallel and produces the conversation's working memory plus the first output token; it performs much arithmetic per byte moved, so it is **compute-bound**, and its duration is the wait before anything appears. **Decode** writes the answer one token at a time: for each token the machine must read all the weights and the conversation's working memory from high-bandwidth memory and do little arithmetic, so it is **memory-bandwidth-bound** and the arithmetic units mostly wait. The source argues that nearly every later technique is a response to this difference.

概念界定 DEFINITION

键值缓存（工作记忆） *Key–value cache (working memory)*

模型读取每个词元时，会在每一层为其计算一对键向量与值向量，供后续词元查询。保留这些向量即可避免重复计算；一个对话全部键值的集合就是它的工作记忆。其大小决定一台机器能同时容纳多少对话，进而决定吞吐与成本。

When a model reads a token it computes a key and a value vector for it at every layer, which later tokens consult. Keeping them avoids recomputation; the collection for a conversation is its working memory. Its size decides how many conversations fit on a machine, and therefore throughput and cost.

2.2 四个数字

The Four Numbers

底本主张，评判一个推理服务只需要四个数字，书中的一切技术都是在不毁坏其他三个的前提下改善其中之一。本研究将其作为全部发现的统一因变量。值得强调的是**有效吞吐**（goodput）与原始吞吐的区别：后者几乎可以通过加大批量无限提高，代价是时延失控；前者只计入同时满足两项时延目标的请求。一个高吞吐、低有效吞吐的服务，只是在忙于生产迟到的答案。

The source holds that an inference service is judged by four numbers, and every technique in the book improves one without ruining the others. This study adopts them as the common dependent variables for all findings. The distinction between **goodput** and raw throughput deserves emphasis: the latter can be raised almost without limit by batching harder while latency slips; the former counts only requests that met both latency targets. A service with high throughput and low goodput is busy producing late answers.

表 2-1 评判推理服务的四个数字
Table 2-1 *The four numbers that judge an inference service*

指标 Measure	含义 What it captures	主要影响因素 Main drivers
首词元时间（TTFT） Time to first token	从请求到达到首个词元离开；用户感知的响应性 Arrival to first token out; felt as responsiveness	排队、预填充、缓存命中 Queueing, prefill, cache hits
每词元时间（TPOT） Time per output token	首词元之后的平均间隔；其倒数即单用户速度 Interval after the first; inverse is speed per user	批量大小、内存带宽、推测解码 Batch size, bandwidth, speculation
有效吞吐 Goodput	每秒完成且满足两项时延目标的请求数 Requests per second meeting both targets	调度、准入控制、阶段干扰 Scheduling, admission, phase interference
单位有效结果成本 Cost per accepted outcome	全部成本除以达到质量标准的结果数 Full cost over results meeting the quality bar	利用率、服务模式、模型与格式 Utilisation, mode, model and format

来源 Source: Cuofano (2026), Chapter IV; table structure by this study.

四个数字沿同一条曲线权衡：以单用户速度为横轴、每加速器吞吐为纵轴，每种配置都是一个点，最优配置构成「前沿」。底本的方法论起点因此是**先定目标，再找前沿**；没有书面目标的服务，会被「谁抱怨得最大声」来回调优。

The four numbers trade along one curve: plot throughput per accelerator against speed per user and each configuration is a point, the best forming a frontier. The source's starting point is therefore **targets first, then the frontier**; a service without written targets is tuned by whoever complains loudest.

2.3 一次请求的生命周期
The Life of a Request

图 2-1 请求生命周期中的六个工程环节
Figure 2-1 *Six engineering steps in the life of a request*



来源 Source: condensed by this study from the eight stations of Cuofano (2026), Part II (admission, batching, prefix cache, separated phases, faster decoding, shrinking, parallelism, cache tiers).

2.4 五层分析框架
A Five-Layer Analytical Framework

本研究据此构建五层分析框架，下层约束上层：内存约束决定哪些站点技术有效，站点选择决定机队容量，利用率与采购方式决定单位成本，而治理层的目标、套件与仪表决定系统朝哪个方向被优化。

On these concepts the study builds a five-layer framework in which lower layers constrain upper ones: the memory limit decides which station techniques work, station choices set fleet capacity, utilisation and purchasing set unit cost, and governance — targets, suite, meters — decides what the system is optimised for.

图 2-2　本研究的五层分析框架
Figure 2-2　*The study's five-layer analytical framework*

治理层 Governance	书面目标、冻结评测套件、七项仪表	Written targets, frozen suite, seven meters
经济层 Economics	单位成本公式、利用率交叉点、能耗	Unit-cost formula, utilisation crossover, energy
机队层 Fleet	硬件、引擎、容量规划、服务模式	Hardware, engines, capacity, serving modes
站点层 Stations	准入、批处理、缓存、分离、推测、压缩	Admission, batching, cache, separation, speculation, shrinking
物理层 Physics	两阶段瓶颈与键值缓存的内存约束	Two-phase bottlenecks and the memory limit of the cache

来源 Source: this study's framework, built on the part structure of Cuofano (2026).

第 3 章 · CHAPTER 3

03 研究设计与方法

Research Design and Method

本研究对底本进行结构化分析性阅读，以机制为分析单位，以可证伪假设组织证据。

中文

The study performs a structured analytical reading of the source, takes mechanisms as its unit of analysis and organises evidence around falsifiable hypotheses.

ENGLISH

3.1 资料与分析单位

Material and Units of Analysis

研究资料为底本全书，包括三十六章正文、尾声及八个附录。本研究区分三类证据：一是**机制性论证**，如两阶段瓶颈与缓存容量的推导；二是**示意算例**，底本以一个 700 亿参数的通用模型和一台八加速器、约 1,128 GB 高带宽内存、每小时全成本 40 美元的通用机器展开计算，并以虚构企业 Aldgate Legal 演示端到端改造；三是**时点测量**，即截至 2026 年第三季度的市场、价格、硬件与基准数据。三类证据的效力不同，本文在使用时分别标注。

The material is the full source volume: thirty-six chapters, an epilogue and eight appendices. The study distinguishes three kinds of evidence: **mechanistic arguments**, such as the derivations of the two-phase bottleneck and cache capacity; **illustrative worked examples**, built on a generic 70-billion-parameter model and a generic eight-accelerator machine with about 1,128 GB of high-bandwidth memory at $40 an hour all-in, plus an invented firm, Aldgate Legal; and **dated measurements** of markets, prices, hardware and benchmarks as of Q3 2026. They carry different weight and are labelled accordingly.

分析单位是「机制」而非产品：每项技术被拆解为它所缓解的约束、它改变的指标、它的成本以及它失效的条件。比较维度则是第二章的四个数字。这种设计使发现可以跨越具体厂商和时点而迁移——底本自己也强调，模型会多次更替，但两个阶段、工作记忆、批处理、缓存以及速度与成本之间的曲线不会改变。

The unit of analysis is the mechanism, not the product: each technique is decomposed into the constraint it relieves, the measure it moves, its cost and the conditions under which it fails. Comparisons run along the four numbers of Chapter 2. This design lets findings travel across vendors and dates — as the source itself stresses, models will change many times, but the two phases, the working memory, the batch, the cache and the curve between speed and cost will not.

3.2 研究假设

Hypotheses

底本附录 A 为其承重主张附上了「证伪条件」。本研究将其整理为五项假设，作为第四至六章的检验框架。下表同时列出每项假设的证伪条件与本文对应章节。

Appendix A of the source attaches falsifiers to its load-bearing claims. The study restates them as five hypotheses that frame Chapters 4 to 6. The table lists each hypothesis with its falsifier and the chapter where it is examined.

表 3-1 研究假设及其证伪条件
Table 3-1 *Hypotheses and their falsifiers*

假设 Hypothesis	证伪条件 Falsified if	章节 Ch.
H1 解码成本由内存流量决定，故取决于批量与缓存大小 H1 Decode cost is set by memory traffic, hence by batch and cache size	实测每词元解码成本对批量和上下文长度不敏感 Measured decode cost per token is insensitive to batch and context length	4 4
H2 缓存命中率是代理工作负载成本的主导杠杆 H2 Cache hit rate is the dominant lever on agent cost	在现实命中率下，代理的输入成本相对输出成本很小 Agent input cost is small relative to output at realistic hit rates	5 5
H3 目标下的有效吞吐而非峰值吞吐预测单位有效结果成本 H3 Goodput at target, not peak throughput, predicts cost per accepted outcome	按峰值调优的服务与按有效吞吐调优的服务成本相当 Peak-tuned services match goodput-tuned ones on that cost	6 6
H4 专用算力仅在利用率超过某阈值时优于按词元付费，且阈值随目标收紧而上升 H4 Dedicated capacity beats pay-per-token only above a threshold that rises as targets tighten	专用算力在低利用率或严格目标下仍更便宜 Dedicated stays cheaper at low utilisation or tight targets	6 6
H5 任何精度或规模削减都须在工作负载自身套件上验证 H5 Every precision or size reduction must be validated on the workload's own suite	公共基准能可靠预测特定工作负载的质量损失 Public benchmarks reliably predict workload-specific loss	4 4

来源 Source: restated from Cuofano (2026), Appendix A.

方法说明 METHOD NOTE

检验方式与边界 *How hypotheses are examined, and the limits*

本研究不进行新的实测，而是检验底本的机制推导是否自洽、示意算例是否支持其结论、时点测量是否与机制方向一致。因此，结论的强度是「机制上成立且与已报告数据一致」，而非「经独立实验证实」。底本作者亦明言：在相信书中任何数字之前，先测量自己的系统。

The study runs no new measurements. It checks whether the source's mechanistic derivations are internally consistent, whether the worked examples support its conclusions, and whether dated measurements point in the direction the mechanisms predict. Conclusions are therefore of the strength 'mechanistically sound and consistent with reported data', not 'independently verified by experiment'. The author himself advises measuring your own system before believing any number in the book.

第 4 章 · CHAPTER 4

04 发现一：工作记忆而非权重决定批处理上限

Finding 1: Working Memory, Not Weights, Sets the Ceiling on Batching

解码受限于内存；批处理是推理最大的经济杠杆，而键值缓存的大小决定了这根杠杆能拉多远。

中文

Decode is bound by memory; batching is the largest economic lever in inference, and the size of the key–value cache decides how far that lever can be pulled.

ENGLISH

4.1 批处理的算术

The Arithmetic of Batching

底本的示意算例：700 亿参数、每参数 1 字节的模型有 70 GB 权重，分布于合计带宽约 38 TB/s 的八个加速器上。为单个对话生成一个词元至少需读完 70 GB，约 1.8 毫秒，即每秒最多约 550 个词元，算力大多空闲。若 64 个对话一同解码，权重每步只读一次、全体共享：每步延长到约 4 毫秒，但总产出升至每秒约 15,800 个词元。

The source's worked example: a 70-billion-parameter model at one byte per parameter has 70 GB of weights on eight accelerators with about 38 TB/s of combined bandwidth. One token for one conversation means reading all 70 GB — at least 1.8 ms — so a single conversation gets at most about 550 tokens a second while arithmetic idles. Decode sixty-four together and the weights are read once per step and shared: each step stretches to about 4 ms, but total output rises to about 15,800 tokens a second.

~0.33MB
每词元缓存（示意模型）
cache per token, illustrative

10.7GB
3.2 万词元对话的工作记忆
working memory, 32k-token chat

77→155
缓存降至 1 字节后的并发对话
concurrent chats, 2→1 byte cache

~30×
批量 64 时的总吞吐提升
throughput gain at batch 64

示意 Illustrative: Cuofano (2026), Chapters II–III, generic 70B model on a generic eight-accelerator machine.

吞吐提升近三十倍，而单用户速度从约每秒 550 词元降至约 250。这正是推理的核心权衡：**只有许多对话共享每次权重读取，解码才便宜；每多一个对话，所有对话都慢一点**。示意模型每词元缓存约三分之一兆字节，七个长对话的工作记忆就超过模型本身；剩余约 830 GB 内存只能容纳约 77 个 3.2 万词元对话，把缓存精度从 2 字节降到 1 字节，容量即翻倍至约 155 个。

Throughput rose almost thirtyfold while each user's speed fell from about 550 to about 250 tokens a second. This is the central trade-off: **decode is cheap only when many conversations share each read of the weights, and each extra conversation slows them all.** The illustrative model needs about a third of a megabyte of cache per token, so seven long conversations outweigh the model; the remaining ~830 GB holds about seventy-seven 32,000-token conversations, doubling to about 155 when the cache is stored at one byte instead of two.

研究发现 FINDING

发现一（支持 H1） *Finding 1 (supports H1)*

决定批处理上限的是工作记忆而非权重，而批处理决定成本。因此，内存容量与带宽——而非峰值算力——才是推理成本最关键的硬件参数；这也解释了 2026 年内存价格、富内存芯片和更小数值格式为何主导了整个领域。

Working memory, not the weights, sets the ceiling on batching, and batching sets cost. Memory capacity and bandwidth — not peak arithmetic — are therefore the hardware parameters that matter most, which explains why memory prices, memory-rich chips and smaller number formats dominated the field in 2026.

4.2 由同一约束衍生的技术
Techniques Derived from the Same Constraint

推测解码：把闲置算力变成速度 *Speculative decoding: idle arithmetic becomes speed*

由于解码受内存所限，大模型一次验证多个候选词元的代价与验证一个相差无几。小型起草器先猜若干词元，大模型一次验证，输出完全一致。接受率 0.8、起草 4 个时，每次验证约产出 3.4 个词元，净加速约 2.4 倍。收益取决于接受率，且在大批量时收缩。

Because decode is memory-bound, verifying several drafted tokens costs the large model little more than one. A small drafter guesses ahead; the large model checks in one pass, and output is identical. At 0.8 acceptance and four drafted tokens a pass yields about 3.4 tokens, a net speed-up of about 2.4×. The gain hinges on acceptance and shrinks at large batches.

量化：更少的字节，但须对照套件 *Quantisation: fewer bytes, checked against the suite*

每减少一个字节都会加速解码。8 位浮点已成常规；主流 GPU 厂商的 4 位格式约为 16 位的 3.5 分之一，自测关键任务精度损失不超过 1%。但公共基准上损失 1% 的格式，在算术、小语种或长上下文检索上可能损失更多。这支持 H5：**只在工作负载自己的冻结套件上压缩**，并比较单位有效结果成本。

Every byte removed speeds decode. 8-bit floating point is routine; the leading GPU maker's 4-bit format is about 3.5 times smaller than 16-bit, losing one percent or less on key tasks in its own tests. Yet a format losing one percent on a public benchmark may lose far more on arithmetic, rare languages or long-context retrieval. This supports H5: **shrink only against the workload's own frozen suite**, comparing cost per accepted outcome.

阶段分离：冲突时才是答案 *Phase separation: an answer only to conflict*

预填充要算力，解码要带宽，同机运行即相互干扰。分块预填充可以缓和；分离部署则在一个机池读取、经高速网络传输工作记忆、在另一机池写出，研究报告在满足两项目标时可服务数倍请求。但它需要高速互联与足够流量，对小型部署得不偿失。

Prefill wants arithmetic and decode wants bandwidth, so on one machine they interfere. Chunked prefill softens this; disaggregation reads on one pool, ships the working memory over a fast network and writes on another, and research found several times more requests served within both targets. But it needs a fast interconnect and enough traffic, and rarely pays for small deployments.

图 4-1　何时分离预填充与解码：一个决策矩阵
Figure 4-1　*When to separate prefill and decode: a decision matrix*



来源 Source: this study's synthesis of the rule of thumb in Cuofano (2026), Chapter VIII.

风险提示 CAUTION

技术不是默认值　*Techniques are not defaults*

本章每项技术都附有失效条件：分离在小规模下增加复杂度，推测在大批量下收益收缩，量化在特定任务上可能悄然损害质量。把它们当作「一律开启」的默认设置，正是第八章所述多类失效的来源。

Every technique in this chapter carries a failure condition: separation adds complexity at small scale, speculation's gain shrinks at large batches, and quantisation can quietly damage quality on specific tasks. Treating them as switch-on-everywhere defaults is the source of several failures described in Chapter 8.

第 5 章 · CHAPTER 5

05 发现二：缓存命中率是代理时代的首要成本杠杆

Finding 2: Cache Hit Rate Is the Primary Cost Lever of the Agent Era

代理把推理变成了「反复阅读」；谁能让前缀保持稳定，谁就能以四分之一的成本完成同一任务。

中文

Agents turn inference into re-reading; whoever keeps the prefix stable can finish the same task at a quarter of the cost.

ENGLISH

5.1 从写作问题到阅读问题

From a Writing Problem to a Reading Problem

多数请求并非从零开始：聊天服务每条消息都附带同样的系统提示，检索服务把同一文档送给许多问题，代理则每一轮都把全部历史连同一个新步骤重新发送。代理的上下文逐步增长，每一步都要重读；生产级代理的输入输出比接近一百比一。对这类负载而言，几乎全部成本和大部分首词元时间都花在阅读输入上。底本引述构建者的判断：缓存命中率——每一步输入中已被读过的比例——是生产环境中最重要的单一指标。

Most requests do not begin from nothing: a chat service sends the same system prompt with every message, a retrieval service sends one document to many questions, and an agent resends its whole history plus one new step at every turn. An agent's context grows step by step and is re-read each time; production agents run at input-to-output ratios near a hundred to one. For such workloads almost all of the cost, and most of the time to first token, is spent reading input. The source cites builders who named the cache hit rate — the share of each step's input already read — the single most important production metric.

供给侧已有对应机制：前缀缓存在新请求命中已知前缀时直接跳到新内容。截至 2026 年 9 月，三家最大的模型 API 都把缓存命中输入定价为新输入的十分之一或更低，条目在数分钟到一小时内过期。

The supply side is ready: prefix caching lets a request that starts with a known prefix skip to the new part. By September 2026 the three largest model APIs priced cached input at a tenth of fresh input or less, with entries expiring after minutes to an hour.

≤0.1×
缓存命中输入的相对价格
relative price of cached input

6×
前缀树缓存的吞吐增益（上限）
prefix-tree throughput gain, up to

15×
缓存分层层的吞吐增益（上限）
cache-tier layer gain, up to

来源 Source: Cuofano (2026), Chapters VII and XII; vendor- and project-reported figures, Q3 2026.

5.2 同一代理，四倍成本差

Same Agent, a Fourfold Cost Gap

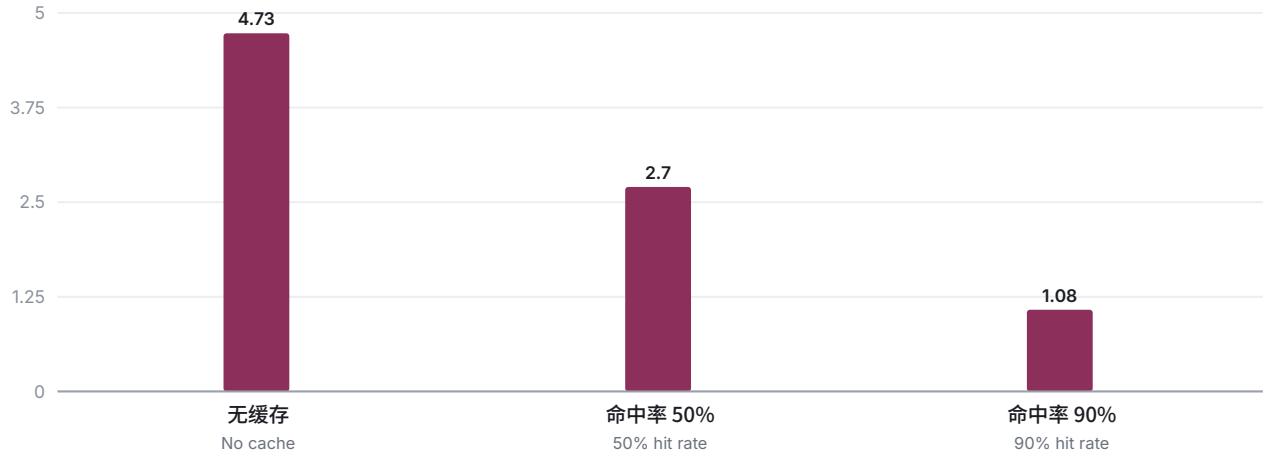
底本的示意算例最能说明问题。设一个代理运行 30 步，每步重读 5 万词元上下文、写出 500 词元；输入每百万词元 3 美元，缓存输入 0.30 美元，输出 15 美元。不使用缓存时，单次任务约 4.73 美元，几乎全部花在重读上；命中率 90% 时约 1.08 美元；若代理框架习惯性改写上下文开头、命中率只有 50%，则约 2.70 美元。同一代理、

The source's worked example makes the point. An agent runs thirty steps, re-reading a 50,000-token context and writing 500 tokens at each; input costs $3 per million tokens, cached input $0.30 and output $15. With no caching the task costs about $4.73, almost all of it re-reading. At a 90% hit rate it costs about $1.08. If the harness habitually edits the start of its context and achieves only 50%, it costs about $2.70. Same agent,

同一模型、同一答案，成本在 1.08 美元到 4.73 美元之间浮动，差别只在于框架是否保持前缀稳定。

same model, same answer: anywhere from $1.08 to $4.73, depending only on whether the harness keeps its prefix stable.

图 5-1 同一代理任务在不同缓存命中率下的成本（美元）
Figure 5-1 *Cost of one agent task at different cache hit rates (USD)*



示意 Illustrative: Cuofano (2026), Chapter VII — 30 steps, 50k-token context, 500 output tokens per step; $3 / $0.30 / $15 per million.

研究发现 FINDING

发现二（支持 H2） *Finding 2 (supports H2)*

在百比一的输入输出比下，输入成本远大于输出成本，H2 的证伪条件不成立。命中率由请求如何构造决定，而非由引擎决定：它同时降低成本与首词元时间，是代理工作负载中投入最少、回报最大的杠杆。

At a hundred-to-one input-to-output ratio, input cost dwarfs output cost, so H2's falsifier does not hold. The hit rate is decided by how requests are built, not by the engine; because it lowers cost and time to first token together, it is the lever with the least effort and the largest return for agent workloads.

5.3 为缓存而设计
Designing for the Cache

1. 按「系统指令—工具定义—参考资料—历史—新步骤」的顺序组织上下文；
2. 历史只追加、不改写；需要遗忘时，按计划大步截断，而非逐步整理；
3. 会话内工具列表保持固定，提示顶部不放时间戳或随机标识；
4. 缓存寿命应长于步骤间隔；自建集群按缓存位置路由请求；
5. 按工作负载设定命中率目标，把命中率下降当作事故处理。

1. Order the context as system instructions, tool definitions, reference material, history, then the new step;
2. Append history, never rewrite it; when material must go, drop it in large planned steps, not by tidying;
3. Keep the tool list fixed within a session, and keep timestamps and random IDs away from the top;
4. Choose cache lifetimes longer than the gap between steps; on self-hosted fleets, route to where the cache sits;
5. Set a hit-rate target per workload and treat a drop as an incident.

5.4 工作记忆成为数据资产
Working Memory Becomes a Data Asset

任一时刻，加速器上的工作记忆大多属于未被服务的对话。缓存分层把闲置工作记忆逐级下沉到主机内存、固态盘与网络存储，需要时取回——对长前缀而言远比重新预填充便宜。2026 年初，主流 GPU 厂商宣布了专用的工作记忆存储层。本研究认为，工作记忆已成为需要容量规划、淘汰策略与成本核算的**数据层**，遵循数据库缓冲池的逻辑。

At any moment most working memory on an accelerator belongs to conversations not being served. Tiering moves it down to host memory, SSDs and networked storage and back on resume — far cheaper than a fresh prefill for long prefixes. In early 2026 the leading GPU maker announced a dedicated storage tier for working memory. The study reads this as working memory becoming a **data tier** with capacity planning, eviction and cost, following the logic of a database buffer pool.

> **稳定内容在前，新内容在后。让缓存保持温热。**
>
> *Stable content first, new content last. Keep the cache warm.*
>
> — Cuofano, Inference Engineering (2026)

第 6 章 · CHAPTER 6

06 发现三：单位成本取决于目标、利用率与服务模式

Finding 3: Unit Cost Depends on Target, Utilisation and Serving Mode

不附时延目标的每词元价格不是价格，只是一条陡峭曲线上的一个点。

A price per token quoted without a latency target is not a price; it is one point on a steep curve.

中文　　ENGLISH

6.1 一个除法，三项决策

One Division, Three Decisions

在自控算力上，词元的单位成本是一个除法：每小时算力成本 ÷（目标下每小时可产出词元 × 利用率）。分子不只是加速器，还包括主机、电力、冷却、网络、缓存分层的存储、运维人员、故障冗余，以及几年内就会被新一代淘汰的硬件折旧。分母中的「目标下的产出」即第二章的有效吞吐，它随时延目标急剧变化；利用率则由流量形态与容量策略决定。三个项各对应一项决策。

On capacity you control, the cost of a token is one division: hourly cost of the capacity ÷ (tokens it can produce per hour at the target × utilisation). The numerator is more than the accelerator: host, power, cooling, networking, storage for the cache tier, the engineers who run the fleet, spare capacity, and depreciation of hardware a newer generation will outclass within a few years. Output at the target is the goodput of Chapter 2 and moves steeply with the latency target; utilisation is set by traffic shape and capacity policy. Each term hides a decision.

以底本的示意机器为例（每小时全成本 40 美元，8,000 词元上下文，缓存 1 字节）：批量 64 时单用户约每秒 250 个词元，成本约每百万词元 0.70 美元；若要求单用户每秒 400 个词元，批量须缩至约 19，成本升至约 1.45 美元；放宽到约每秒 190 个词元、批量 96，成本降至约 0.60 美元；批量为 1 时则超过 20 美元。**同一机器、同一模型，成本仅因服务速度不同就相差近 2.5 倍。**

Take the source's illustrative machine ($40 an hour all-in, 8,000-token contexts, one-byte cache). At a batch of sixty-four each user gets about 250 tokens a second and the cost is about $0.70 per million tokens. Require 400 tokens a second per user and the batch shrinks to about nineteen, raising the cost to about $1.45. Relax to about 190 tokens a second with a batch of ninety-six and it falls to about $0.60. At a batch of one it exceeds $20. **The same machine and model differ by nearly 2.5× in cost purely because of how fast each user is served.**

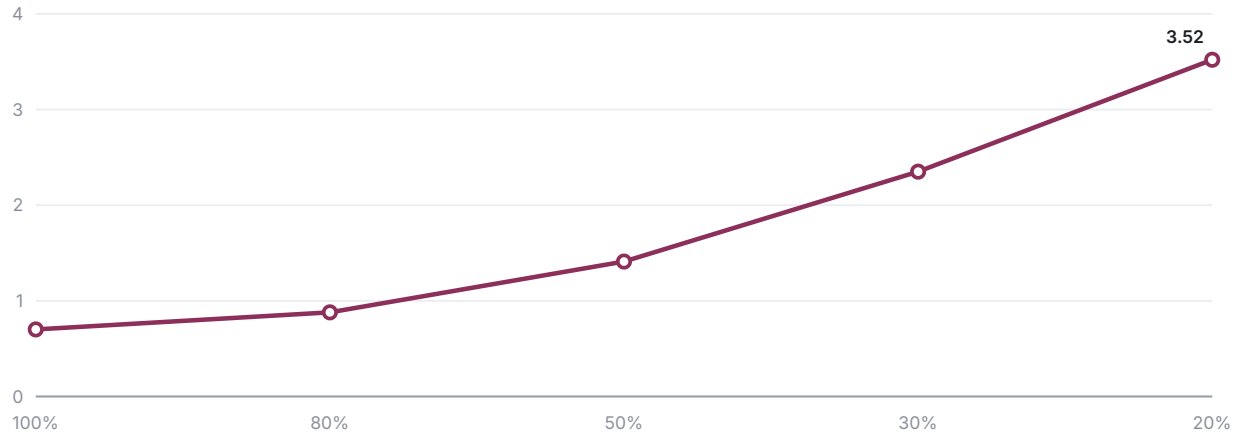
$0.60	$0.70	$1.45	>$20
约 190 词元/秒（批量 96） ~190 tok/s, batch 96	约 250 词元/秒（批量 64） ~250 tok/s, batch 64	约 400 词元/秒（批量约 19） ~400 tok/s, batch ~19	批量为 1 batch of one

示意 Illustrative: cost per million output tokens at full utilisation; Cuofano (2026), Chapter XXVI, generic machine at $40/hour.

利用率是乘数。上述 0.70 美元在 50% 利用率下变为 1.41 美元，在 30% 时变为 2.35 美元。底本的结论是：在 20% 利用率下精心调优的配置，单位成本高于在 80% 利用率下表现平平的配置。这支持 H3——决定单位成本的是目标下的有效吞吐与真实利用率，而非峰值吞吐。

Utilisation is a multiplier. The $0.70 becomes $1.41 at 50% utilisation and $2.35 at 30%. The source concludes that the best-tuned configuration at twenty percent utilisation costs more per token than a mediocre one at eighty percent. This supports H3: unit cost is decided by goodput at the target and by real utilisation, not by peak throughput.

图 6-1 利用率对每百万词元成本的影响（美元）
Figure 6-1 *Effect of utilisation on cost per million tokens (USD)*



示意 Illustrative: 100%, 50% and 30% points from Cuofano (2026), Chapter XXVI; 80% and 20% points derived by this study from the same formula.

6.2 利用率交叉点
The Utilisation Crossover

算力有两种商业形态：按词元付费，由服务商吸收闲置；按小时付费，由买方吸收闲置。二者的分界由一个比率决定：交叉利用率 = 专用算力小时成本 ÷（按词元替代方案单价 × 目标下满载每小时词元数）。按底本援引的示意数据（每小时 40 美元、满载每小时 1.44 亿混合词元、同一开源模型托管价每百万 0.40 美元），交叉点约为 69%：低于此，托管 API 更便宜；高于此，专用机器更便宜。

Capacity comes in two commercial forms: pay by the token and let the provider absorb idle time, or pay by the hour and absorb it yourself. One ratio divides them: crossover utilisation = hourly cost of dedicated capacity ÷ (pay-per-token price × tokens per hour at full load at the target). With the illustrative figures the source borrows — $40 an hour, 144 million blended tokens an hour at full load, and $0.40 per million for the same open-weight model through a managed API — the crossover sits at about 69%. Below it the managed API is cheaper; above it the dedicated machine is.

底本在此基础上做了三点修正，本研究认为它们构成对 H4 的完整支持。第一，**目标移动交叉点**：目标越紧，分母越小，交叉点越高；需要单用户每秒 400 词元的负载，可能要接近满利用率专用算力才划算，而托管服务商能在同一机器上混合快慢流量，这是单一买方无法复制的优势。第二，**低谷可以填充**：白天跑交互、夜间跑批处理的机队可以跨过任一负载单独无法跨过的交叉点。第三，**溢出改变问题**：专用算力按稳定基线配置，峰值通过网关按词元购买。

The source adds three refinements, which together this study takes as full support for H4. First, **the target moves the crossover**: a tighter target shrinks the denominator and raises the crossover, so a workload needing 400 tokens a second per user may need near-full utilisation before dedicated capacity pays — while a managed provider can mix fast and slow traffic on the same machines, an advantage no single buyer can copy. Second, **troughs can be filled**: a fleet running interactive work by day and batch by night can clear a crossover neither clears alone. Third, **spillover changes the question**: size dedicated capacity for the steady base and buy the peak by the token through the gateway.

6.3 六种服务模式：二十倍价差
Six Serving Modes, a Twentyfold Spread

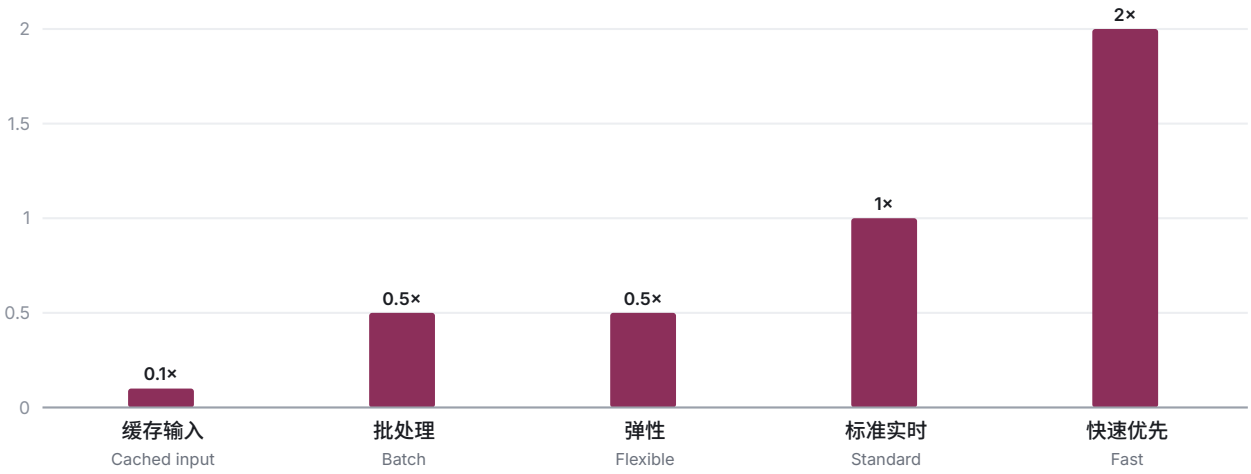
2026 年市场以六种模式出售同样的词元：缓存输入（≤0.1× 标价）、批处理（0.5×）、弹性（约 0.5×，可能被拒）、标准实时（1×）、快速 / 优先（约 2×）与按时

The 2026 market sells the same tokens in six modes: cached input (≤0.1× list), batch (0.5×), flexible (~0.5×, may be refused), standard real-time (1×), fast or priority (~2×) and provisioned throughput billed by

计费的预留吞吐。同一词元最便宜与最昂贵处理方式的价差超过二十倍，但多数企业仍几乎全部走标准模式。本研究认为，服务模式是工作负载的**架构属性**而非采购细节：缓存模式买不到，只能靠提示结构「赚取」。

time. The spread between the cheapest and dearest way to process a token exceeds twenty times, yet most enterprises still send nearly everything through the standard mode. The study treats the mode as an **architectural property** of the workload: the cached mode cannot be bought, only earned by prompt structure.

图 6-2 六种服务模式的相对价格（标价 = 1）
Figure 6-2 *Relative price of the serving modes (list = 1)*



来源 Source: Cuofano (2026), Chapter XVI, September 2026 converged grid; cached input is an upper bound (≤0.1×); provisioned is billed by time and omitted.

研究发现 FINDING

发现三（支持 H3、H4） *Finding 3 (supports H3 and H4)*

单位成本是时延目标、利用率与服务模式的乘积式结果。目标应由用户或下游流程的真实需要设定——阅读中的人无法从超过阅读速度的输出中获益，而处于关键路径上的代理可能需要每一毫秒。

Unit cost is a multiplicative outcome of latency target, utilisation and serving mode. The target should be set by what the user or downstream process actually needs: a reading person gains nothing above reading speed, while an agent on the critical path may need every millisecond.

6.4 推理与代理：按任务而非按词元定价
Reasoning and Agents: Price per Task, Not per Token

推理模型以思考词元换质量，代理以反复阅读换自主性，二者都使成本单位从词元转向任务。底本建议把推理强度设为仍能通过套件的最低档，并为整个任务设定时延预算，只在关键路径上使用快速档——两秒的等待对人是一次，对三十步的代理则是三十次。

Reasoning models trade thinking tokens for quality and agents trade re-reading for autonomy; both move the unit of cost from token to task. The source sets reasoning effort to the lowest level that passes the suite and gives the whole task a latency budget, spending fast-tier capacity only on the critical path — a two-second wait is paid once by a person but thirty times by a thirty-step agent.

第 7 章 · CHAPTER 7

07 案例研究与比较分析

Case Study and Comparative Analysis

一个虚构的法律科技企业演示了三项发现如何叠加；服务商版图则揭示了买方真正在为什么付费。

中文

An invented legal-technology firm shows how the three findings compound; the provider map shows what a buyer is really paying for.

ENGLISH

7.1 案例：Aldgate Legal 的两个季度

Case: Two Quarters at Aldgate Legal

案例观察 CASE OBSERVATION

案例设定 *Case set-up*

Aldgate Legal 是底本为教学而虚构的合同审查平台，运行三类工作负载：每日 4 万次交互式合同问答（每次约 2 万词元输入）；每日 3,000 份合同、每份 25 步的审查代理（每步约 6 万词元输入）；每晚 5 万份文档的条款抽取。起初全部以标准实时模式调用前沿模型，月度模型成本约 45.4 万美元。所有数字均为示意。

Aldgate Legal is a contract-review platform the source invents for teaching. It runs three workloads: 40,000 interactive contract questions a day (about 20,000 input tokens each); a review agent processing 3,000 contracts a day at 25 steps each (about 60,000 input tokens per step); and nightly clause extraction over 50,000 documents. Initially all called a frontier model in standard real-time mode, for about $454,000 a month. All figures are illustrative.

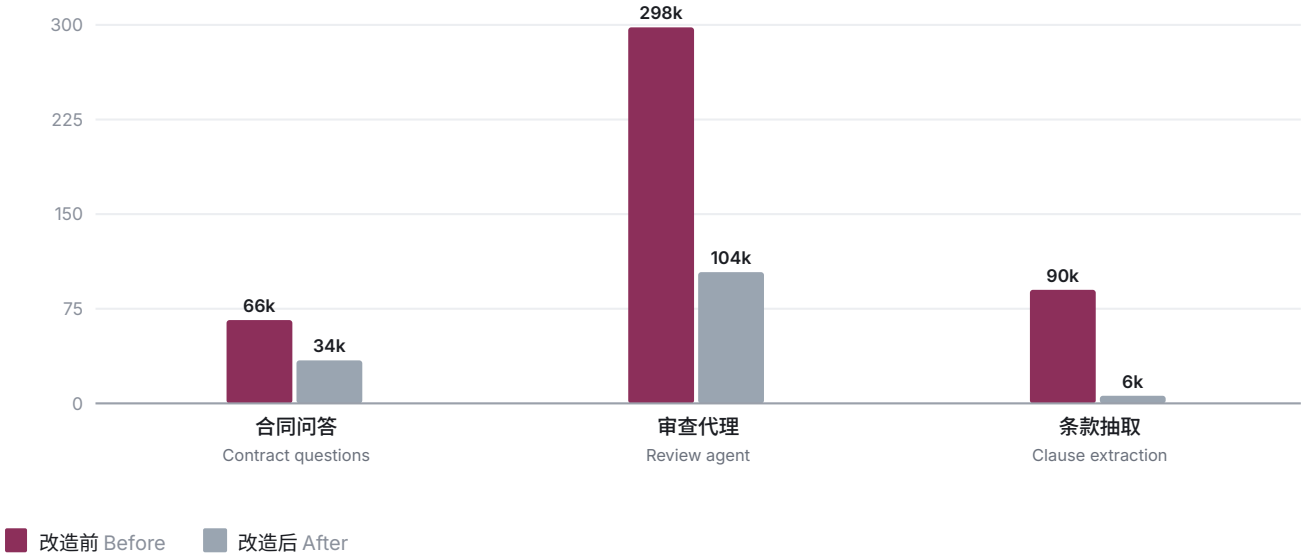
诊断显示问题出在请求构造：问答服务把用户名和时间戳放在合同之上，命中率仅 20%；审查代理每隔几步改写历史，命中率 35%，每份合同约 3.31 美元；本可等待的条款抽取以实时模式运行。

Diagnosis located the problem in how requests were built: the question service put the user's name and a timestamp above the contract (20% hit rate); the review agent rewrote its history every few steps (35%, about $3.31 per contract); and extraction that could wait ran in real time.

第一季度只动缓存：把用户名与时间戳移到合同之下，问答命中率升至 70%；代理框架改为只追加、固定工具列表、一小时缓存寿命，命中率升至 88%，每份合同约 1.16 美元，落入 1.50 美元预算。第二季度调整模式与模型：以 2,000 份评分文档冻结抽取套件，托管小型开源模型验收率与前沿模型相差不到半个百分点，改走批处理后月成本约 6,000 美元；每月约 28,800 美元的专用节点则被交叉点算术否决。约三分之二的节省未更换前沿模型、未谈判价格。

Quarter one touched only the cache: moving the name and timestamp below the contract raised the question hit rate to 70%; an append-only harness with a fixed tool list and one-hour lifetimes raised the agent's to 88%, about $1.16 per contract, inside its $1.50 budget. Quarter two changed mode and model: a frozen suite of 2,000 graded documents cleared a small open-weight model within half a point of the frontier, and extraction moved to batch for about $6,000 a month; a dedicated node at about $28,800 a month was rejected by the crossover arithmetic. Two-thirds of spend went without changing the frontier model or negotiating a price.

图 7-1 Aldgate Legal 各工作负载月度模型成本（千美元）
Figure 7-1 *Aldgate Legal monthly model cost by workload (USD thousands)*



示意 Illustrative: Cuofano (2026), Appendix C; invented composite, rounded figures, 30-day months.

$454k→$144k
月度模型成本
monthly model cost

−68%
总降幅，验收率不变
total reduction, same acceptance

35%→88%
审查代理缓存命中率
review-agent hit rate

0
购置的硬件
hardware bought

示意 Illustrative: Cuofano (2026), Appendices B–C.

7.2 比较分析：谁在供应词元
Comparative Analysis: Who Serves the Tokens

底本把供应方归为八类：模型厂商、超大规模云平台、新型云、推理专业厂商、芯片厂商、开源引擎项目、路由与网关、以及测量者。其中最有解释力的一条规则是：**每个供应方出售其所控制的那部分服务栈，并把其余部分的价格打包进去**。下表从「控制什么、对买方意味着什么」的角度做了比较。

The source sorts suppliers into eight kinds: model makers, hyperscaler platforms, neoclouds, inference specialists, chip makers, open-source engine projects, routers and gateways, and measurers. Its most explanatory rule is that **every provider sells the part of the serving stack it controls and prices the rest into it.** The table compares them by what each controls and what that means for a buyer.

表 7-1 推理服务版图：控制点与买方含义
Table 7-1 *The serving map: control points and buyer implications*

供应方类型 Provider type	最擅长控制 Controls best	对买方的含义 Implication for the buyer
模型厂商 Model makers	模型与价格网格 The model and the price grid	把价格网格当设计说明书；各家缓存机制不同 Read the grid as a design brief; cache mechanics differ
超大规模云平台 Hyperscaler platforms	承诺消费与既有生态 The commitment and the neighbourhood	最易购买，最难看清单位有效结果成本 Easiest to buy, hardest to see cost per outcome
新型云与推理专业厂商 Neoclouds and specialists	容量、价格或单用户速度 Capacity, price or speed per user	开源模型最便宜；可承接关键路径步骤 Cheapest open-weight tokens; can take critical-path steps
芯片厂商 Chip makers	模型与硬件之间的软件层 The layer between model and metal	把「十倍降本」视为待自测的假设 Treat '10× cheaper' as a hypothesis to test
开源引擎 Open-source engines	所有人都在运行的技术 The techniques everyone runs	服务层锁定的保险；须钉住版本 Insurance against lock-in; pin versions
路由网关与测量者 Routers and measurers	每次请求由谁服务；比较曲线 Who serves each request; the curves	自有网关策略与日志；基准只是线索 Own the gateway policy; benchmarks are leads

来源 Source: this study's comparison drawn from Cuofano (2026), Part IV; positions as read in Q3 2026.

> **每个供应方出售其所控制的那部分服务栈，并把其余部分的价格打包进去。**
>
> *Every provider sells the part of the serving stack it controls, and prices the rest into it.*
>
> — Cuofano, Inference Engineering (2026), Part IV

第 8 章 · CHAPTER 8

08 讨论：失效模式、局限与风险

Discussion: Failure Modes, Limitations and Risks

推理服务的十类失效有一个共同根源：度量的是机器，而不是答案。

The ten ways inference services fail share one root: they measure the machine instead of the answer.

中文　　ENGLISH

8.1 十类失效及其共同根源

Ten Failures and Their Common Root

底本识别了十种失效，每种都有先于损害出现的症状。本研究将其归为六组：在每一组中，都是某个机器指标看似健康，而面向答案的指标在恶化。

The source identifies ten failures, each with a symptom visible before the damage. The study groups them into six clusters; in each, a machine-facing measure looks healthy while an answer-facing one deteriorates.

表 8-1 推理失效的六组归类
Table 8-1 *Six clusters of inference failure*

组别 Cluster	失效模式 Failure modes	可见症状 Visible symptom	补救 Remedy
目标与度量 Targets & measures	无目标调优；有吞吐无有效吞吐 Tuned without targets; throughput without goodput	配置随投诉变化；吞吐与投诉同涨 Configs change after escalations; output and complaints both rise	先定目标；有效吞吐为首要指标 Targets first; goodput as headline
缓存 Cache	冷缓存；无视缓存的路由 Cold cache; cache-blind routing	输入账单随步数增长；自建命中率偏低 Input bill grows per step; low self-hosted hit rate	缓存友好结构；按缓存位置路由 Cache-friendly prompts; cache-aware routing
调度 Scheduling	卡顿的流式输出 The stuttering stream	上传文档时流速骤降 Streaming collapses on uploads	分块预填充，再考虑分离 Chunked prefill, then separation
质量 Quality	破坏任务的压缩；无上限的思考者 The shrink that broke the task; the unbounded thinker	人工更正增加；单价降而单任务成本升 More human corrections; cost per task rises as price falls	对照套件压缩；按套件设推理强度 Shrink against the suite; effort from the suite
容量与采购 Capacity & buying	闲置机队；标准模式默认 The idle fleet; the standard-mode default	单价远高于托管价；账单几无批处理与缓存词元 Unit cost far above managed price; few batch or cached tokens	交叉点、填谷、溢出；按负载标注模式 Crossover, trough-filling, spillover; tag modes
运维 Operations	未钉住版本的引擎 The unpinned engine	无人部署却出现回退 A regression nobody deployed	钉住版本，经套件晋升 Pin versions; promote through the suite

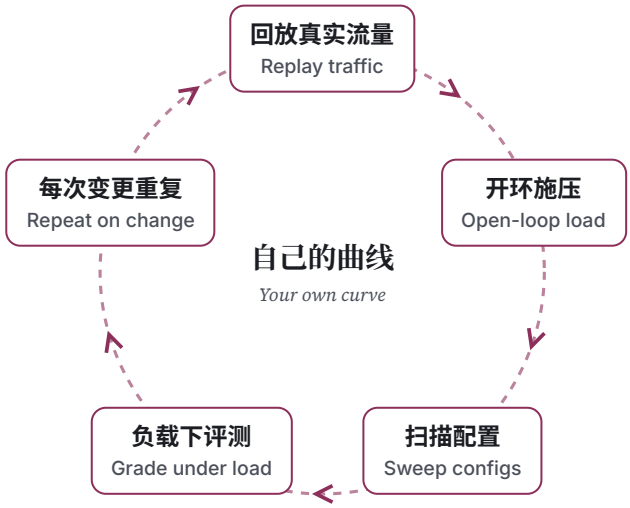
来源 Source: Cuofano (2026), Chapter XXIX; clustering by this study.

8.2 治理启示：测量自己的曲线
Governance Implication: Measure Your Own Curve

既然失效源于度量错位，治理的核心就是正确的测量回路：回放真实流量；以开环施压（闭环测试会在系统变慢时自动减压，使其显得更好）；扫描配置并记录第 95 百分位时延、有效吞吐与单位有效结果成本；在负载下运行冻结套件；每次变更后重复。

If failure stems from mismeasurement, governance centres on the right loop: replay real traffic; load in an open loop (a closed loop eases off as the system slows and flatters it); sweep configurations, recording 95th-percentile latencies, goodput and cost per accepted outcome; run the frozen suite under load; and repeat on every change.

图 8-1 测量自身曲线的闭环
Figure 8-1 *The loop for measuring your own curve*



来源 Source: Cuofano (2026), Chapter XVIII.

8.3 研究局限
Limitations

- **时效性**：数据截至 2026 年第三季度，引擎约两周一发布；结论依赖机制而非数字。
- **示意与自报**：核心算术取整且忽略真实损耗；能耗与「十倍降本」多为厂商自报；案例企业为虚构。
- **单一底本**：本研究未对照独立的生产数据集。
- **风险**：开源网关库曾于 2026 年 3 月遭包仓库投毒；关键负载需在网关保留第二路由；代理上下文增长会使上月达标的批量失守。

- **Timeliness**: data date to Q3 2026 and engines ship every two weeks; conclusions rest on mechanisms, not numbers.
- **Illustrative and self-reported data**: core arithmetic is rounded and ignores real losses; energy and '10× cheaper' claims are vendor-reported; the case firm is invented.
- **Single source**: no independent production dataset was consulted.
- **Risks**: a gateway library suffered a package-registry compromise in March 2026; critical workloads need a second gateway route; growing agent contexts can push last month's compliant batch off target.

第 9 章 · CHAPTER 9

09 结论与未来研究方向

Conclusions and Future Research Agenda

模型会多次更替，但两个阶段、工作记忆、批处理、缓存与速度—成本曲线不会改变。

Models will change many times; the two phases, the working memory, the batch, the cache and the speed–cost curve will not.

中文　　ENGLISH

9.1 对研究问题的回答

Answers to the Research Questions

就 RQ1 而言，单位成本由物理约束（解码受内存所限、工作记忆决定批量）与三项工程决策（时延目标、利用率、服务模式）共同决定；专用与托管的分界是一个随目标收紧而上升、可借填谷与溢出下移的利用率交叉点，稳健形态是「专用承担基线、按词元承担峰值」。就 RQ2 而言，各项技术都回应同一内存约束且都有条件：阶段分离只在目标冲突且规模足够时有效，推测解码在小批量、高接受率时有效，量化只在通过套件时有效，而缓存设计几乎在所有代理负载中都有效。

On RQ1, unit cost is set by physical constraints (decode bound by memory; working memory setting the batch) and three decisions (latency target, utilisation, serving mode); the boundary between dedicated and managed capacity is a utilisation crossover that rises as targets tighten and falls with trough-filling and spillover, the robust form being dedicated for the base, pay-per-token for the peak. On RQ2, every technique answers the same memory constraint and each is conditional: separation pays only when targets conflict at scale, speculation at modest batches with high acceptance, quantisation only when it passes the suite — while cache design pays in almost every agent workload.

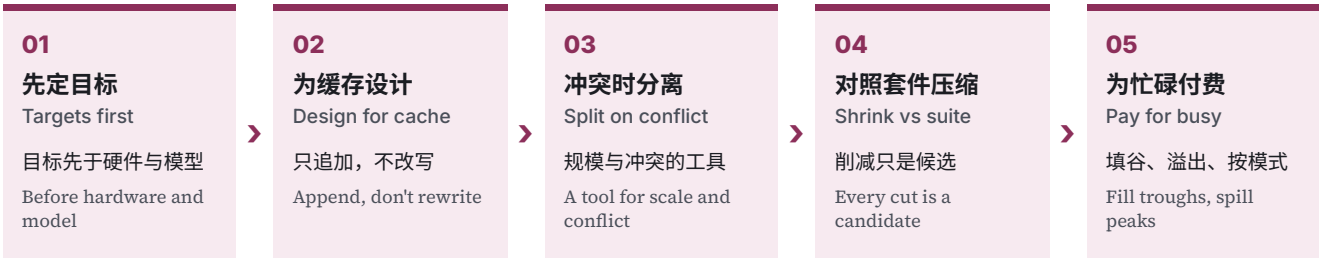
就 RQ3 而言，应以七项仪表按负载、按趋势度量服务：有效吞吐、第 95 百分位的首词元与每词元时间、缓存命中率、单位有效结果成本及模式构成、目标下的专用算力利用率、推测接受率、单位有效结果能耗；并刻意排除不附时延的峰值吞吐、设备报告的「GPU 繁忙率」等在服务变差时反而可能改善的指标。

On RQ3, the service should be graded per workload and on trend by seven meters: goodput; 95th-percentile time to first token and per token; cache hit rate; cost per accepted outcome with its mode mix; utilisation of dedicated capacity at the target; speculation acceptance; and energy per accepted outcome — deliberately excluding measures such as peak throughput without a latency or device-reported 'GPU busy', which can improve as the service worsens.

9.2 五项行动原则

Five Moves

图 9-1 由三项发现导出的五项行动原则
Figure 9-1 *Five moves derived from the three findings*



来源 Source: Cuofano (2026), Part VII.

> **按目标服务。让缓存保持温热。为忙碌付费，而非为空闲付费。**
>
> *Serve to the target. Keep the cache warm. Pay for busy, not idle.*
>
> — Cuofano, Inference Engineering (2026)

9.3 更广的意义：词元成为公用事业

The Wider Meaning: The Token Becomes a Utility

底本以电力作比：当价格、可靠性与计量单位标准化，客户便不再关心发电机。词元正走在同一条路上——API 形态标准化，价格网格共享，技术写进开源引擎，产能以电站规模建设。在公用事业化的市场中，优势转向理解自身负载的客户：何时达峰、什么可等、什么可复用、每单位产出值多少。

The source draws on electricity: once price, reliability and units standardise, customers stop thinking about generators. Tokens are on the same path — a standard API shape, a shared price grid, techniques built into open engines, capacity at power-station scale. In a utility market the advantage moves to customers who understand their own load: when it peaks, what can wait, what can be reused and what each unit is worth.

9.4 未来研究议程

Future Research Agenda

- **独立验证**：以真实生产轨迹检验五项假设，尤其是 H2 在不同代理架构下的稳健性与 H4 交叉点随目标移动的实际斜率。
- **缓存经济学**：工作记忆成为存储层后，其淘汰策略、跨租户共享的安全边界与定价模型尚缺系统研究。
- **任务级度量**：建立跨服务商可比的单位有效结果成本与能耗口径。
- **组织维度**：目标由谁签署、质量标准由谁负责，如何影响十类失效的发生率。
- **纵向追踪**：在后续版本的时点数据中复测本文所引数字，检验机制的预测力。

- **Independent validation**: test the five hypotheses on real production traces, especially H2's robustness across agent architectures and the actual slope of H4's crossover as targets move.
- **Cache economics**: once working memory is a storage tier, its eviction policy, the security boundary of cross-tenant sharing and its pricing lack systematic study.
- **Task-level measures**: build a comparable basis for cost and energy per accepted outcome across providers.
- **Organisation**: how ownership of targets and quality standards affects the incidence of the ten failures.
- **Longitudinal tracking**: re-take the cited figures in later dated readings to test the predictive power of the mechanisms.

参考文献与研究说明 *References & Research Notes*

研究说明 *Note on Method and Sources*

本研究是对底本《推理工程》（Cuofano，2026）的结构化分析性阅读，而非译本或摘要。文中引用的市场与产品数据均为底本所载、截至 2026 年第三季度的测量值；底本中以虚构模型、虚构机器或虚构企业（Aldgate Legal）给出的算例，本文一律标注为「示意」。个别图表在底本公式基础上做了推算，亦已在注释中说明。所有数字都应视为机制的实例，而非对任何产品的规格陈述。

This study is a structured analytical reading of the source volume, Inference Engineering (Cuofano, 2026); it is neither a translation nor a summary. Market and product figures are the source's own measurements, dated to Q3 2026. Worked examples that the source builds on a generic model, a generic machine or an invented company (Aldgate Legal) are marked illustrative throughout. Where a chart extends the source's formulas, the note says so. All numbers should be read as instances of mechanisms, not as specifications of any product.

参考文献 *References*

1. Cuofano, G. (2026). *Inference Engineering — How a Model Becomes a Dependable, Economical Service*. The Business Engineer Library. businessengineer.ai
2. Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS*.
3. Yu, G. et al. (2022). Orca: A Distributed Serving System for Transformer-Based Generative Models. *OSDI*.
4. Dao, T. et al. (2022). FlashAttention; FlashAttention-3 (2024); FlashAttention-4 (2026).
5. Kwon, W. et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP*.
6. Leviathan, Y., Kalman, M. & Matias, Y. (2022). Fast Inference from Transformers via Speculative Decoding; Chen, C. et al. (2023). Accelerating Large Language Model Decoding with Speculative Sampling; Li, Y. et al. (2025). EAGLE-3.
7. Zheng, L. et al. (2023). SGLang: Efficient Execution of Structured Language Model Programs.
8. Patel, P. et al. (2024). Splitwise. *ISCA*; Zhong, Y. et al. (2024). DistServe. *OSDI*; Qin, R. et al. (2025). Mooncake. *FAST*.
9. NVIDIA (2025–26). Dynamo; Introducing NVFP4; BlueField-4 context-memory storage; Q2 fiscal 2027 results.
10. MLCommons (2026). MLPerf Inference v6.0 results; SemiAnalysis (2025–26). InferenceMAX and InferenceX.
11. Google (2025–26). Measurement of the environmental footprint of AI inference (August 2025); TPU 8t and 8i (April 2026); I/O 2026 token volumes.
12. Epoch AI (2025–26). Analyses of inference prices, output length, context windows and cost per task; Andreessen Horowitz (2024). LLMflation.
13. Manus (2025). Context Engineering for AI Agents.
14. Provider pricing documentation for caching, batch, flexible, fast and provisioned modes (read September 2026).

AI 时代商业工程 · 课题研究系列 · Business Engineering in the AI Era · Research Study Series
No. 07 / 12 · 2026